CONNECT THE DOTS: NAVIGATING NETWORKS

Session 1: Introduction to Graph Theory, BFS, and DFS Algorithms

Course: Computer Science Fundamentals • Topic: Graph Traversal • Level: Beginner / Intermediate

1. Introduction: Small World Phenomena & Social Networks

Have you ever wondered how you are connected to a famous celebrity, a world leader, or someone living on another continent through just a few mutual friends? This fascinating concept is known as the **"Six Degrees of Separation"** or the **Small World Phenomenon**.

We live in a massively interconnected world. Every person you know is connected to other people, who are in turn connected to many others. In computer science, we model these real-world interconnections using a mathematical structure called a **Graph**.

2. What is a Graph? Core Concepts

A **Graph** is a collection of points (called **Nodes** or **Vertices**) connected by lines (called **Edges**). Graphs allow computer scientists to model relationships and solve complex navigation, routing, and networking problems.

Nodes (Vertices)

Nodes represent individual entities, points, or objects in a network.

- **Delhi Metro:** Stations (e.g., Rajiv Chowk, Hauz Khas)
- **Social Media:** User Profiles
- **Family Tree:** Individual Family Members

Edges (Connections)

Edges represent the relationships, links, or pathways connecting two nodes.

- **Delhi Metro:** Train tracks connecting stations
- **Social Media:** Friendships or Following links
- **Family Tree:** Parent-Child relationships

> **Real-World Example: The Delhi Metro Graph**
> Imagine traveling on the Delhi Metro network. Key interchange points like Rajiv Chowk and Hauz Khas are **NODES**. The physical railway tracks running between these stations are **EDGES**. Every time you board a train and travel to the next station, you are navigating along an edge from one node to another!

3. Graph Traversal Algorithms: Meet Buddy BFS & Daring DFS

When a computer searches for an item, a path, or a person inside a graph, it uses a systematic process called **Graph Traversal**. The two most fundamental graph traversal algorithms are **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.

Buddy BFS (Breadth-First Search)

Daring DFS (Depth-First Search)

Motto: "I like to check all nearby friends first before going farther!"

Buddy BFS explores a network level by level, spreading outward in concentric circles like ripples in a pond. First, Buddy visits all immediate neighbors (1 step away), then neighbors of those neighbors (2 steps away), and so forth.

- **Data Structure:** Queue (FIFO — First-In, First-Out)
- **Key Feature:** Guarantees shortest path in unweighted networks.
- **Best For:** GPS navigation, finding closest destination.

Motto: "I love to explore one path all the way to the end before trying another!"

Daring DFS is an adventurous explorer. She picks one direction and goes as deep into the network as possible until reaching a dead end. Once stuck, she backtracks to the last intersection and tries a new unexplored branch.

- **Data Structure:** Stack (LIFO — Last-In, First-Out) / Recursion
- **Key Feature:** Explores deep branches fully before returning.
- **Best For:** Solving mazes, topological sorting, games.

4. Comparison Summary: BFS vs. DFS

Feature	BFS (Buddy BFS)	DFS (Daring DFS)
Search Order	Level-by-level (Ripples in a pond)	Branch-by-branch (Deep dive)
Underlying Data Structure	Queue (FIFO)	Stack (LIFO) or Recursion
Shortest Path Finding	Optimal for unweighted graphs	Not guaranteed to find shortest path
Memory Requirement	Higher (stores entire frontier levels)	Lower (stores current search path)
Primary Applications	GPS Navigation, Finding nearest friends	Solving Sudoku/Mazes, Cycle detection

5. Python Implementation Guide: BFS and DFS

Below is the standard Python code for representing graphs using adjacency lists and executing both BFS and DFS traversals.

Representing the Graph in Python

```
# Graph representation using an Adjacency List (Dictionary)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C']
}
```

Breadth-First Search (BFS) Implementation

```
from collections import deque

def bfs(graph, start_node):
    visited = set()
    queue = deque([start_node])
    visited.add(start_node)
    traversal_order = []

    while queue:
        node = queue.popleft()  # FIFO: Remove from the front
        traversal_order.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)  # Enqueue unvisited neighbors

    return traversal_order

# Example Output: bfs(graph, 'A') -> ['A', 'B', 'C', 'D', 'E', 'F']
```

Depth-First Search (DFS) Implementation

```
def dfs_iterative(graph, start_node):
    visited = set()
    stack = [start_node]
    traversal_order = []

    while stack:
        node = stack.pop()  # LIFO: Remove from top of stack
        if node not in visited:
            visited.add(node)
            traversal_order.append(node)
            # Push neighbors in reverse to maintain alphabetical visit order
            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
```

```
                    stack.append(neighbor)

    return traversal_order

# Example Output: dfs_iterative(graph, 'A') -> ['A', 'B', 'D', 'E', 'C', 'F']
```

STUDENT WORKSHEET & PRACTICE ASSESSMENT

Date: _____

Student Name: _____

Question 1: Terminology Identification BASIC

Consider a social media platform like Instagram or LinkedIn.

a) What real-world components represent the **Nodes** in this graph?

b) What real-world interactions or relationships represent the **Edges**?

Question 2: BFS Traversal Execution INTERMEDIATE

Suppose we have an undirected graph with the following connections:
• Node A is connected to Nodes B and C.
• Node B is connected to Nodes D and E.
• Node C is connected to Node F.

Starting at **Node A**, list the exact sequence of nodes visited using **Breadth-First Search (BFS)**. (Assume alphabetical tie-breaking for neighbors).

Sequence of visited nodes: [____ , ____ , ____ , ____ , ____ , ____]

Explanation of steps / Queue state trace:

Question 3: DFS Traversal Execution INTERMEDIATE

Using the exact same graph described in Question 2 (A–B, C; B–D, E; C–F):

Starting at **Node A**, list the exact sequence of nodes visited using **Depth-First Search (DFS)**. (Assume alphabetical tie-breaking for neighbors).

Sequence of visited nodes: [____ , ____ , ____ , ____ , ____ , ____]

Explanation of steps / Stack state trace:

Question 4: Python Code Analysis CODING ASSESSMENT

Examine the Python BFS snippet below and answer the following questions:

```python
from collections import deque

def traverse(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        curr = queue.popleft()
        print(curr, end=" ")
        for nxt in graph[curr]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)
```

a) What specific data structure method ensures that this function processes nodes in First-In, First-Out (FIFO) order?

b) What would happen if we removed line `visited.add(nxt)` and didn't track visited nodes in a graph containing cycles?

Question 5: Algorithmic Selection & Real-World Application ADVANCED

GPS navigation apps like Google Maps or Apple Maps need to find the quickest driving route from your location to a hospital during an emergency.

a) Which algorithm — BFS or DFS — is better suited for this emergency navigation task?

b) Provide two technical reasons justifying your choice based on how the selected algorithm operates:

Reason 1:

Reason 2:

Question 6: Conceptual Analysis & Backtracking ADVANCED

In Depth-First Search (DFS), what is the term used when the algorithm hits a node with no unvisited neighbors and must move backward to try a different branch? Describe how a Stack structure enables this mechanism.

Term: _____

Explanation: